



Social Media Database System

Project Report

Section

D7

Instructor

Asna Abroo

Group Members

Sr. No.	Name	Roll Number
1	Talha Khurram	L1F24BSCS0938
2	Hussain Durrani	L1F24BSCS0937
3	Ali Hamza	L1F24BSCS1037

Project Description

This project models the backend database structure of a social media platform. It covers user management, content sharing, social interactions, messaging, and notifications. The design follows the full database lifecycle — from requirement gathering to ERD design to a normalized relational schema — with a focus on scalability and data integrity.

Features

The system supports the following key features:

- **User Management** — Registration, profile setup (bio, picture, gender, DOB), and account management.
- **Posts** — Text content with privacy settings and creation/update timestamps.
- **Post Media** — Multiple media attachments (images, videos) per post.
- **Likes** — Like support for both posts and comments via separate junction tables.
- **Comments** — Commenting on posts with nested/threaded replies using self-referencing keys.
- **Followers** — Follow/unfollow system modeled as a many-to-many user relationship.
- **Messaging** — Direct messages between users with read/seen tracking.
- **Saved Posts** — Bookmark posts for later viewing.
- **Notifications** — Automatic alerts for likes, comments, follows, and messages.

Entities

The database is built around the following core entities:

- **User** — Central entity storing profile details (username, email, bio, picture, etc.).
- **Post** — User-created content with privacy settings and timestamps.
- **Post Media** — Media attachments (images, videos) linked to a post.
- **Comment** — User comments on posts, with support for nested replies.
- **Follower** — Junction entity capturing follow relationships between users.
- **Message** — Direct messages between two users.
- **Saved Post** — Junction entity linking users to their bookmarked posts.
- **Post Like / Comment Like** — Junction entities tracking likes on posts and comments.
- **Notification** — System alerts triggered by user activities.

Relationships

The relationships between entities are outlined below:

- **User → Post (One-to-Many)** — A user creates multiple posts; each post belongs to one user.
- **Post → Post Media (One-to-Many)** — A post can have multiple media items.
- **User → Comment → Post (Many-to-Many)** — Comments link users to posts.
- **Comment → Comment (Recursive)** — Replies form a threaded tree via parent_comment_id.
- **User ↔ User — Followers (Many-to-Many)** — Follow relationships using follower_id and following_id.
- **User ↔ User — Messaging (Many-to-Many)** — Messages linked by sender_id and receiver_id.
- **User ↔ Post — Saved Posts (Many-to-Many)** — Users bookmark posts.
- **User ↔ Post — Post Likes (Many-to-Many)** — Users like posts with a liked_at timestamp.
- **User ↔ Comment — Comment Likes (Many-to-Many)** — Users like comments.
- **User → Notification (One-to-Many)** — Notifications belong to one user, triggered by another.

Entity-Relationship Diagram (ERD)

The ERD provides a conceptual overview of the database design — all entities, attributes, and relationships.

Key observations:

- User is the central entity, connected to nearly every other entity.
- Recursive relationships exist in Comments (replies) and Followers (user follows user).
- Many-to-many relationships (Likes, Saves, Follows) use junction entities.
- Post Media is a weak entity dependent on Post.

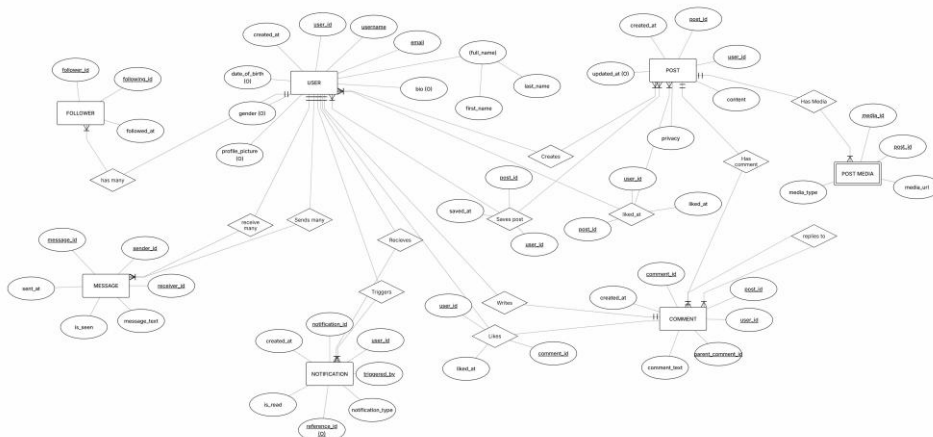


Figure 1: Entity-Relationship Diagram

Relational Model

The relational model converts the ERD into normalized tables with primary keys (PK), foreign keys (FK), and unique constraints (U).

Users Table

- `user_id` — Primary Key
- `username` — Unique
- `email` — Unique
- `password_hash` —
- `full_name` —
- `bio` —
- `profile_picture` —
- `gender` —
- `date_of_birth` —
- `created_at` —

Posts Table

- `post_id` — Primary Key
- `content` —
- `privacy` —
- `created_at` —
- `updated_at` —
- `user_id` — Foreign Key → Users

Post Media Table

- `media_id` — Primary Key
- `media_url` —
- `media_type` —
- `post_id` — Foreign Key → Posts

Comments Table

- `comment_id` — Primary Key
- `comment_text` —
- `created_at` —
- `post_id` — Foreign Key → Posts
- `user_id` — Foreign Key → Users
- `parent_comment_id` — Foreign Key → Comments (self-referencing for nested replies)

Followers Table

- `follower_id` — Foreign Key → Users (composite PK)
- `following_id` — Foreign Key → Users (composite PK)
- `followed_at` —

Messages Table

- **message_id** — Primary Key
- **message_text** —
- **is_seen** —
- **sent_at** —
- **sender_id** — Foreign Key → Users
- **receiver_id** — Foreign Key → Users

Saved Posts Table

- **user_id** — Foreign Key → Users (composite PK)
- **post_id** — Foreign Key → Posts (composite PK)
- **saved_at** —

Post Likes Table

- **user_id** — Foreign Key → Users (composite PK)
- **post_id** — Foreign Key → Posts (composite PK)
- **liked_at** —

Comment Likes Table

- **user_id** — Foreign Key → Users (composite PK)
- **comment_id** — Foreign Key → Comments (composite PK)
- **liked_at** —

Notifications Table

- **notification_id** — Primary Key
- **notification_type** —
- **reference_id** —
- **is_read** —
- **created_at** —
- **user_id** — Foreign Key → Users
- **triggered_by** — Foreign Key → Users

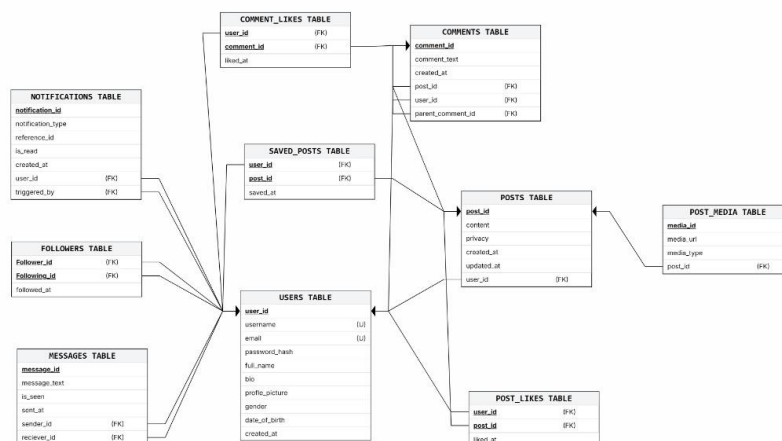


Figure 2: Relational Model Diagram